
.HTACCESS MULTI-DOMAIN + WIDGET ENGINE
PRESERVATION

Necoyoad

v11

*The Apache rewrite rules that power multi-domain multi-
store,
and how to preserve the widget engine in a future
framework*

part I: .htaccess three-layer rewrite architecture
root \$\to\$ /web/ \$\to\$ index.php \$\to\$ store detection
part II: Widget engine preservation strategy
7 core abstractions \$\to\$ Laravel equivalents
API contract, dev experience, non-negotiables
listings: 8 · pages: 14

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

Necoyoad — .htaccess Multi-Domain Routing + Widget Engine Preservation (v11)

*The Apache rewrite rules that power multi-domain multi-store,
and how to preserve the widget engine in a future framework*

Abstract

This is the eleventh and final technical volume. It covers two topics the user identified as critical: (1) the `.htaccess` rewrite rules that integrate with the multi-domain multi-store feature, and (2) the widget engine — the platform’s most valuable architectural asset — and how to preserve its full flexibility when migrating to a modern framework like Laravel.

Part I dissects the three-layer `.htaccess` configuration (`root` → `/web/` → `index.php`) that enables multi-domain routing: the root `.htaccess` rewrites everything to `/web/`, the `web/.htaccess` rewrites to `index.php?_route_`, and `index.php` itself detects the store via URL path, GET parameter, or subdomain regex. Together, these three layers implement a transparent multi-domain multi-store routing system where each domain (or subdomain, or URL path prefix) maps to a different store.

Part II is the widget engine preservation strategy. It identifies the seven core abstractions that make the widget engine flexible (`loadWidgets`, `NecoWidget`, the `{%widget_name%}` substitution, the `module:settings` filter, the `Module` base class, the `deps.php` manifest, and the per-entity/per-item composition), and provides a concrete implementation plan for preserving each in a Laravel or custom-framework rebuild. The goal: the same “drop a widget into a position, configure it in the admin, see it on the page” developer experience, with the same dynamic/manual/hybrid composition modes.

Keywords: Necoyoad, OpenCart, PHP 8.1, `.htaccess`, `mod_rewrite`, multi-domain, multi-store, widget engine, preservation, Laravel, Blade, Livewire, View Composers, dynamic components.

Contents

I	The .htaccess Multi-Domain Multi-Store Routing	3
1	The Three-Layer Rewrite Architecture	3
2	The Root .htaccess	3
2.1	The Rewrite-to-/web/ Rule	4
2.2	The Subdomain Normalisation	4
2.3	Security Rules	5
3	The web/.htaccess	5
3.1	The SEO URL Rewrite Rule	5
4	The Multi-Domain Multi-Store Integration	6
4.1	The Complete Multi-Domain Request Flow	6
4.2	Multi-Domain with Different Document Roots	6
II	The Widget Engine Preservation Strategy	6
5	Why the Widget Engine is the Most Valuable Asset	6
6	The Seven Core Abstractions	7
7	Preservation Strategy: Laravel Implementation	7
7.1	Abstraction 1: loadWidgets() → View Composer	7
7.2	Abstraction 2: NecoWidget → WidgetService	8
7.3	Abstraction 3: {%widget_name%} → Blade Slots	8
7.4	Abstraction 4: module:settings Filter → Event Listener	9
7.5	Abstraction 5: Module Base Class → Blade Component Base	10
7.6	Abstraction 6: deps.php Manifest → Vite Manifest + AssetManifest Service	10
7.7	Abstraction 7: Per-Entity/Per-Item Composition → Query Scopes	11
8	The Developer Experience: Creating a New Widget	11
8.1	Current (Necoyoad)	12
8.2	Future (Laravel)	12
9	What Must NOT Change	12
10	The Widget Engine API Contract	13
11	Conclusion	13
11.1	The .htaccess is a Transparent Multi-Domain Enabler	13
11.2	The Widget Engine is the Platform's Soul	13
11.3	The Preservation Strategy is Concrete	13
11.4	Final Word	14

Part I

The .htaccess Multi-Domain Multi-Store Routing

1 The Three-Layer Rewrite Architecture

Necoyoad uses three layers of URL rewriting to route requests to the correct store:

1. **Root .htaccess** (253 lines) — rewrites all requests to `/web/` (the web root). Also handles www-stripping and subdomain normalisation.
2. **web/.htaccess** (611 lines) — rewrites non-file, non-directory requests to `index.php?_route_`. This is the SEO-URL entry point.
3. **web/index.php** (85 lines) — detects the store via three strategies (URL path, GET param, subdomain) and loads the matching `app/<store>/config.php`.

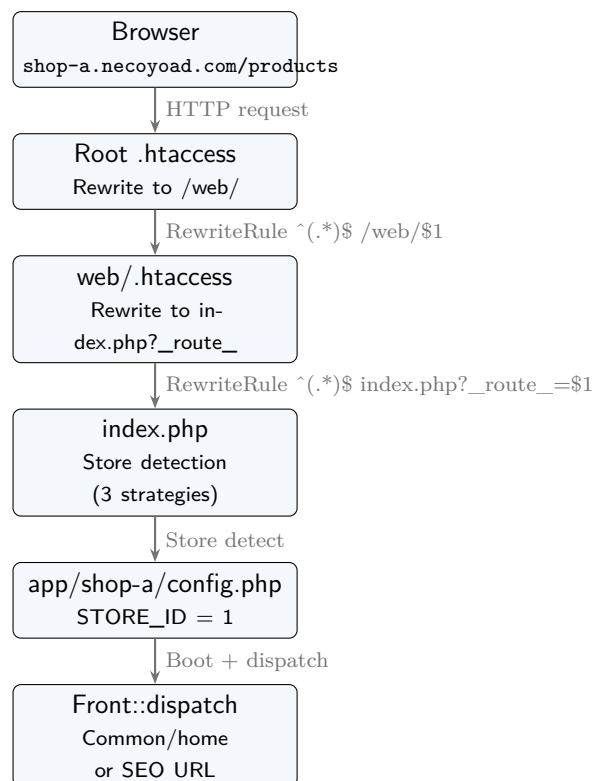


Figure 1: The three-layer rewrite architecture. The root .htaccess rewrites to `/web/`, the web/.htaccess rewrites to `index.php`, and `index.php` detects the store and dispatches.

2 The Root .htaccess

The root .htaccess (253 lines) serves two purposes:

1. **Rewrite to `/web/`:** all requests are internally redirected to the `/web/` subdirectory (where `index.php` lives).
2. **Subdomain normalisation:** strips `www.` and normalises subdomains for the `necoyoad.com` domain.

```

1 <IfModule mod_rewrite.c>
2   Options +FollowSymlinks
3   Options -Indexes
4   RewriteEngine On
5   RewriteBase /
6
7   # Layer 1: Rewrite everything to /web/
8   RewriteCond %{REQUEST_URI} !~/web/
9   RewriteRule ^(.*)$ /web/$1 [L,QSA]
10
11  # Layer 2: SEO URL rewrite (if not a real file/directory)
12  RewriteCond %{REQUEST_FILENAME} !-f
13  RewriteCond %{REQUEST_FILENAME} !-d
14  RewriteRule ^(.*)\?*$ index.php?_route_=$1 [L,QSA]
15 </IfModule>
16
17 # www stripping
18 <IfModule mod_rewrite.c>
19   RewriteCond %{HTTPS} !=on
20   RewriteCond %{HTTP_HOST} ^www\.(.+$) [NC]
21   RewriteRule (.*?) //%{HTTP_HOST}/$1 [R=301,L]
22 </IfModule>
23
24 # Subdomain normalisation (necoyoad.com only)
25 <IfModule mod_rewrite.c>
26   RewriteCond %{HTTPS} !=on
27   RewriteCond %{HTTP_HOST} !~necoyoad.com [NC]
28   RewriteCond %{HTTP_HOST} ^([^.]+\.)necoyoad.com [NC]
29   RewriteRule (.*?) //%1.%{HTTP_HOST}/$1 [R=301,L]
30 </IfModule>
31
32 Options -MultiViews
33 ErrorDocument 404 /index.php?r=error/404

```

Listing 1: Root .htaccess — the rewrite rules (abridged).

2.1 The Rewrite-to-/web/ Rule

```

RewriteCond %{REQUEST_URI} !~/web/
RewriteRule ^(.*)$ /web/$1 [L,QSA]

```

This rule fires when the request URI does *not* already start with `/web/`. It prepends `/web/` to the URI. So a request to `shop-a.necoyoad.com/products` becomes `shop-a.necoyoad.com/web/products` (internally — the browser URL doesn't change). This lets the application live in a subdirectory while appearing to serve from the root.

2.2 The Subdomain Normalisation

```

RewriteCond %{HTTP_HOST} !~necoyoad.com [NC]
RewriteCond %{HTTP_HOST} ^([^.]+\.)necoyoad.com [NC]
RewriteRule (.*?) //%1.%{HTTP_HOST}/$1 [R=301,L]

```

This rule fires when the host is a subdomain of `necoyoad.com` (but not `necoyoad.com` itself). It captures the subdomain label (`%1`) and normalises the URL. This is the Apache-level counterpart of the PHP subdomain regex in `index.php` (v2, v5).

2.3 Security Rules

The root .htaccess also:

- Blocks access to dotfiles (.git, .htaccess itself) via RewriteRule "(.|\./)\." - [F].
- Blocks access to sensitive file extensions (.tpl, .bak, .config, .sql, .ini, .log, .sh, .inc, .swp, .dist).
- Sets session.cookie_httponly true for PHP sessions.
- Disables MultiViews (prevents Apache from content-negotiating, which would interfere with the rewrite rules).

3 The web/.htaccess

The web/.htaccess (611 lines) is the main configuration file. It handles:

- **SEO URL rewrite:** the core rule that sends everything to index.php?_route_=.
- **MIME types:** proper types for JS, fonts, SVG, video, audio.
- **Compression:** mod_deflate for HTML, CSS, JS, JSON, XML, SVG, fonts.
- **Caching:** mod_expires with per-type max-age (1 year for images/fonts, 1 week for CSS/JS, no cache for HTML).
- **CORS:** Access-Control-Allow-Origin "*" for fonts and images.
- **Security:** blocks dotfiles, blocks sensitive extensions, disables directory listing, custom 404/403 error pages.
- **UTF-8:** AddDefaultCharset utf-8 + per-type charset.
- **IE compatibility:** X-UA-Compatible "IE=Edge".
- ETag and Last-Modified header management.

3.1 The SEO URL Rewrite Rule

```
1 <IfModule mod_rewrite.c>
2   Options +FollowSymlinks
3   Options -Indexes
4   RewriteEngine On
5   RewriteBase /
6   RewriteCond %{REQUEST_FILENAME} !-f
7   RewriteCond %{REQUEST_FILENAME} !-d
8   RewriteRule ^(.*)\?*$ index.php?_route_=$1 [L,QSA]
9 </IfModule>
```

Listing 2: web/.htaccess — the SEO URL rewrite rule.

This is the single most important rule in the entire .htaccess. It says: if the request is not for a real file (!-f) and not for a real directory (!-d), rewrite it to index.php?_route_=. The _route_ parameter carries the URL path (e.g. products/my-product), which the PHP ControllerCommonSeoUrl rewriter (v2) then resolves into a controller route via the url_alias table.

The [L,QSA] flags mean: Last (stop processing rules) and Query String Append (preserve any existing GET parameters).

4 The Multi-Domain Multi-Store Integration

The `.htaccess` files don't directly implement multi-store detection — that's done in PHP (`index.php`). But they enable it by:

1. **Transparent `/web/` prefix:** the root `.htaccess` rewrites to `/web/` so the application doesn't need to know it's in a subdirectory. This means all stores share the same `.htaccess` rules regardless of their domain.
2. **Subdomain passthrough:** the `.htaccess` does *not* rewrite subdomains to specific directories. It passes the `HTTP_HOST` header through to PHP, where `index.php` reads it and detects the store via the subdomain regex.
3. **SEO URL passthrough:** the `_route_` parameter carries the full URL path. PHP's `ControllerCommonSeoUrl` then strips the store folder prefix (if present) and resolves the remaining path via `url_alias`.

4.1 The Complete Multi-Domain Request Flow

1. Browser requests `https://shop-a.necoyoad.com/products`.
2. Root `.htaccess`: `REQUEST_URI` does not start with `/web/`, so rewrite to `/web/products`. The `HTTP_HOST` header (`shop-a.necoyoad.com`) is preserved.
3. `web/.htaccess`: `/web/products` is not a real file or directory, so rewrite to `index.php?_route_=products`.
4. `index.php`:
 - (a) Strategy 1 (URL path): splits `products` by `/`, checks each segment against `store.folder`. No match.
 - (b) Strategy 2 (`?store_id`): not set.
 - (c) Strategy 3 (subdomain regex): a `preg_match` call captures the subdomain label from `shop-a.necoyoad.com`, yielding `$matches[1] = 'shop-a'`.
 - (d) `file_exists('app/shop-a/config.php')` → yes.
 - (e) Loads `app/shop-a/config.php` which sets `STORE_ID = 1` and `DIR_APPLICATION = 'app/shop-a/'`.
5. `ControllerCommonSeoUrl`: resolves `products` via `url_alias` → `store/product/all`.
6. `Front::dispatch(new Action('store/product/all'))`.

4.2 Multi-Domain with Different Document Roots

For domains that point to different document roots (not subdomains of `necoyoad.com`), the `.htaccess` doesn't need modification — each domain's document root contains its own copy of the `.htaccess + index.php`. The `index.php` store detection still works via Strategy 1 (URL path folder match) or Strategy 2 (`?store_id` GET parameter).

Part II

The Widget Engine Preservation Strategy

5 Why the Widget Engine is the Most Valuable Asset

The widget engine is the platform's crown jewel. It provides:

- **Drop-in composition:** an admin drags a widget into a position; the widget appears on the page. No code changes.
- **Per-entity overrides:** different widgets on each product, post, page, or category.
- **Per-item composition:** widgets on individual banner slides (v9).
- **Three composition modes:** dynamic (admin-configured), manual (hardcoded placeholders), hybrid.
- **Visual editing:** drag-and-drop via `nt-editable` attributes.
- **Asset management:** per-widget CSS/JS via `deps.php` manifest + route-aware loader.
- **Developer experience:** a new widget module is a controller + a template + an `install.php`. That's it.

No other part of the codebase provides this level of flexibility. The admin CRUD, the marketing pipeline, the multi-store architecture — all are important, but the widget engine is what makes Necoyoad a *page builder*, not just an e-commerce platform.

6 The Seven Core Abstractions

To preserve the widget engine in a future framework, these seven abstractions must be replicated:

Table 1: The seven core widget engine abstractions.

Abstraction	Current impl.	Role
1. <code>loadWidgets()</code>	Controller method	Queries the widget tree from the DB, merges widget routes into <code>\$children</code> .
2. <code>NecoWidget</code>	Helper class (785 lines)	The widget DAO: queries <code>property</code> EAV table, filters by device/lang/store/object.
3. <code>{%widget_name%}</code> substitution	<code>Controller::fetch()</code>	Post-render string replacement of placeholder tokens with widget HTML.
4. <code>module:settings</code> filter	Hook on Controller	The seam where each widget module injects its data (banner items, product list, etc.).
5. Module base class	<code>system/classes/module.php</code>	Auto-loads <code>deps.php</code> assets for the widget's route.
6. <code>deps.php</code> manifest	Theme asset files	Declares which CSS/JS files apply to which routes.
7. Per-entity / per-item composition	Session <code>object_type</code> + <code>NecoWidget::getWidgets()</code>	Widgets scoped to a specific product, post, page, category, or banner item.

7 Preservation Strategy: Laravel Implementation

7.1 Abstraction 1: `loadWidgets()` → View Composer

Preserving `loadWidgets` as a View Composer

In Laravel, `loadWidgets()` becomes a View Composer that runs before every page render. The composer:

1. Reads the current route, store, language, device, and session `object_type`.

2. Queries the widget tree (via the WidgetService, see below).
3. For each widget, instantiates its Blade component class and captures the rendered HTML.
4. Publishes the widget HTML into the view as `$widgets[$position][$widgetName]`.
5. The Blade template iterates `$widgets` and emits `<x-dynamic-component>` tags or raw HTML.

The key insight: `loadWidgets()` is a *view data populator*, not a rendering mechanism. In Laravel, View Composers serve exactly this role — they populate view data before the view renders. The widget tree structure (rows, columns, widgets) is preserved as a nested array in the view data.

7.2 Abstraction 2: NecoWidget → WidgetService

Preserving NecoWidget as a WidgetService

The NecoWidget helper class (785 lines) becomes a WidgetService Laravel service:

```
class WidgetService
{
    public function __construct(
        private WidgetRepository $widgets,
        private StoreContext $store,
        private LanguageContext $language,
    ) {}

    public function getTree(string $position, ?string $objectType
        = null, ?int $objectId = null): array
    {
        return $this->widgets->queryTree(
            storeId: $this->store->id(),
            languageId: $this->language->id(),
            position: $position,
            objectType: $objectType,
            objectId: $objectId,
        );
    }
}
```

The query changes from LIKE substring matching to a proper JSON column query or a junction table. The device/auth/route/landing-page filters are preserved as query conditions, not string-substring matches.

7.3 Abstraction 3: {%widget_name%} → Blade Slots

Preserving the placeholder substitution

The `{%widget_name%}` substitution becomes Blade's `@stack` / `@push` or dynamic components.

Option A: Dynamic Blade Components

```
{{-- In the page template --}}
@foreach ($widgets['main'] as $row)
```

```

        @foreach ($row['columns'] as $column)
            @foreach ($column['widgets'] as $widget)
                <x-dynamic-component
                    :component="$widget['component']"
                    :settings="$widget['settings']"
                />
            @endforeach
        @endforeach
    @endforeach
@endforeach

```

Option B: Stack-based (for manual composition)

```

{{-- In the page template (manual hardcoding) --}}
<div class="hero">
    @stack('hero_banner')
</div>
<div class="features">
    @stack('product_overview_1')
    @stack('product_overview_2')
</div>

{{-- In each widget's Blade component --}}
@push('hero_banner')
    <x-widgets.banner :banner="$banner" />
@endpush

```

Both options preserve the three composition modes:

- **Dynamic:** Option A — the View Composer populates `$widgets`, the template iterates.
- **Manual:** Option B — the template uses `@stack`, the widget uses `@push`.
- **Hybrid:** both in the same template.

7.4 Abstraction 4: `module:settings` Filter → Event Listener

Preserving the `module:settings` filter

The `module:settings` filter (where each widget module injects its data) becomes a Laravel Event/Listener:

```

// In the Banner widget's service provider
Event::listen(WidgetSettingsEvent::class, function ($event) {
    if ($event->widget->module === 'banner') {
        $banner = Banner::with('items.descriptions')
            ->findOrFail($event->settings['banner_id']);
        $event->data['banner'] = $banner;
        $event->template =
            "widgets.banner.{ $banner->jquery_plugin }";
    }
});

```

The event fires for each widget before rendering. The listener checks the module name, loads the widget's data, and sets the template. This preserves the original's per-module injection pattern while using Laravel's event system.

7.5 Abstraction 5: Module Base Class → Blade Component Base

Preserving the Module base class

The Module base class (which auto-loads `deps.php` assets) becomes a base Blade Component:

```
abstract class WidgetComponent extends Component
{
    public function __construct(
        public array $settings,
        public ?object $widget = null,
    ) {
        // Auto-load this widget's assets (deps.php equivalent)
        app(AssetManifest::class)->loadForWidget(static::class);
    }

    // Subclasses override this to inject their data
    abstract public function data(): array;

    public function render()
    {
        return view($this->template, $this->data());
    }
}
```

Each widget module becomes a Blade Component that extends `WidgetComponent`. The constructor auto-loads assets (the `deps.php` equivalent). The `data()` method replaces the `module:settings` filter — each widget returns its own data. The `render()` method uses the dynamic template (per-entity or per-jquery_plugin).

7.6 Abstraction 6: `deps.php` Manifest → Vite Manifest + AssetManifest Service

Preserving the `deps.php` manifest

The `deps.php` manifest (which declares which CSS/JS files apply to which routes) becomes a Vite manifest + a Laravel `AssetManifest` service:

```
class AssetManifest
{
    private array $loaded = [];

    public function loadForWidget(string $widgetClass): void
    {
        $widgetName = Str::kebab(class_basename($widgetClass));
        $manifest = $this->getManifest();

        foreach ($manifest as $asset => $routes) {
            if ($routes === '*' ||
                in_array("module/{$widgetName}", $routes)) {
                if (!in_array($asset, $this->loaded)) {
                    $this->loaded[] = $asset;
                    // Push to Vite's tag resolver or Laravel Mix
                    app('asset')->enqueue($asset);
                }
            }
        }
    }
}
```

```

    }
}

```

The manifest file changes from PHP to JSON or YAML, but the concept is the same: declare which assets apply to which routes, and the service loads matching assets for the current route.

7.7 Abstraction 7: Per-Entity/Per-Item Composition → Query Scopes

Preserving per-entity and per-item widget composition

The per-entity widget override (where product pages show product-specific widgets) becomes a query scope on the `WidgetRepository`:

```

class WidgetRepository
{
    public function queryTree(
        int $storeId,
        int $languageId,
        string $position,
        ?string $objectType = null,
        ?int $objectId = null,
    ): array {
        // Default tree (no object filter)
        $tree = WidgetRow::with('columns.widgets')
            ->where('store_id', $storeId)
            ->where('position', $position)
            ->where('landing_page', 'all')
            ->orWhere('landing_page', $this->currentRoute())
            ->orderBy('sort_order')
            ->get();

        // Per-object override (if objectType/objectId provided)
        if ($objectType && $objectId) {
            $objectWidgets = WidgetRow::with('columns.widgets')
                ->where('store_id', $storeId)
                ->where('position', $position)
                ->where('object_type', $objectType)
                ->where('object_id', $objectId)
                ->orderBy('sort_order')
                ->get();

            $tree = $tree->merge($objectWidgets);
        }

        return $tree->toArray();
    }
}

```

This preserves the two-query merge (default tree + per-object tree) and the `only:` prefix (skip the default tree, load only the per-object tree) by simply not running the first query when `only` is set.

8 The Developer Experience: Creating a New Widget

8.1 Current (Necoyoad)

1. Create `app/admin/controller/module/<name>/widget.php` (admin config form).
2. Create `app/shop/controller/module/<name>.php` (storefront controller, extends `Module`).
3. Create `app/shop/view/theme/choroni/module/<name>.tpl` (storefront template).
4. Create `install.php` + `uninstall.php`.
5. Add CSS/JS to `web/assets/theme/<theme>/css,js/deps.php`.
6. Done. The admin can now drag the widget into any position.

8.2 Future (Laravel)

1. Create `app/Filament/Pages/Widgets/<Name>Settings.php` (admin config form, using Filament or Laravel Nova).
2. Create `app/View/Components/Widgets/<Name>.php` (extends `WidgetComponent`).
3. Create `resources/views/components/widgets/<name>.blade.php` (storefront template).
4. Create a migration for the widget's default settings.
5. Add CSS/JS to the Vite manifest.
6. Done. The admin can now drag the widget into any position.

The developer experience is essentially the same: a controller (component), a template, a config form, and asset registration. The key difference is that the Laravel version uses Blade components instead of raw PHP templates, and Eloquent instead of raw SQL — but the *concept* is identical.

9 What Must NOT Change

Non-negotiable: the widget composition experience

The following aspects of the widget engine *must* be preserved in any rebuild:

1. **The admin can compose pages without code.** A non-technical user drags widgets into positions, configures them via a form, and the page renders. This is the number-one feature.
2. **Three composition modes.** Dynamic (admin-configured), manual (hardcoded placeholders in templates), and hybrid (both in the same template).
3. **Per-entity overrides.** Different widgets on each product, post, page, category, or banner item.
4. **The `module:settings` seam.** Each widget module has a single, well-defined injection point where it loads its data and selects its template.
5. **The asset manifest.** CSS/JS files are declared in a manifest and auto-loaded for matching routes. The developer doesn't manually enqueue assets.
6. **The visual editor hooks.** `nt-editable` (or equivalent) attributes on every widget wrapper, so a visual editor can identify and manipulate widgets.
7. **Per-entity template override.** Each entity can use a different template, set via the admin, stored in EAV (or JSON column in Laravel).

10 The Widget Engine API Contract

To ensure the widget engine survives a framework migration, here is the API contract that any future implementation must satisfy:

Table 2: The widget engine API contract.

Capability	Contract
Query widgets by position	<code>getTree(position, objectType?, objectId?)</code> returns a nested array of rows, columns, widgets.
Register a widget module	A class that extends <code>WidgetComponent</code> + a Blade template + a settings form.
Inject widget data	A method (or event listener) that receives the widget settings and returns the data + template path.
Render a widget	The widget's Blade component renders its template with the injected data.
Compose dynamically	A View Composer populates <code>\$widgets[\$position]</code> before the page renders.
Compose manually	Templates can use <code>@stack</code> / <code>@push</code> or <code><x-dynamic-component></code> to hardcode widget positions.
Per-entity override	<code>getTree()</code> accepts <code>objectType</code> + <code>objectId</code> and merges per-object widgets into the default tree.
Per-item composition	Same as per-entity, with <code>objectType = 'banner_item'</code> (or any other item type).
Asset loading	An <code>AssetManifest</code> service loads CSS/JS for the current route + widget modules.
Visual editor hooks	Every widget wrapper emits <code>data-widget</code> , <code>data-position</code> , <code>data-row</code> , <code>data-column</code> attributes.
Template override	Each entity can specify a template path (stored as a property/JSON column), resolved as: entity template → config default → hardcoded fallback.

11 Conclusion

11.1 The .htaccess is a Transparent Multi-Domain Enabler

The `.htaccess` configuration (root + web) is a transparent multi-domain enabler. It doesn't implement store detection — it passes the `HTTP_HOST` and URL path through to PHP, where `index.php` does the actual detection. The three-layer architecture (root rewrite → web rewrite → PHP dispatch) allows the application to live in a subdirectory while appearing to serve from the root, and allows multiple domains to share the same codebase with different stores.

11.2 The Widget Engine is the Platform's Soul

The widget engine is not just a feature — it is the platform's soul. It is what makes Necoyoad a page builder, not just an e-commerce platform. The seven core abstractions (`loadWidgets`, `NecoWidget`, the `{%widget_name%}` substitution, the `module:settings` filter, the `Module` base class, the `deps.php` manifest, and the per-entity/per-item composition) together form a flexible composition system that allows non-technical users to build complex pages by dragging widgets into positions, and developers to create new widgets with minimal boilerplate.

11.3 The Preservation Strategy is Concrete

The preservation strategy is not abstract — each of the seven abstractions has a concrete Laravel equivalent:

- `loadWidgets()` → View Composer
- `NecoWidget` → `WidgetService`
- `{%widget_name%}` → Blade dynamic components / `@stack`
- `module:settings` → Event listener

- `Module` base → `WidgetComponent` base
- `deps.php` → Vite manifest + `AssetManifest`
- Per-entity composition → Query scopes on `WidgetRepository`

The developer experience — “create a controller, a template, a config form, and register assets” — remains the same. The framework changes; the concept doesn’t.

11.4 Final Word

The Necoyoad widget engine is a genuinely innovative piece of architecture. It anticipated by years the page-builder patterns that WordPress’s block editor, Drupal’s Layout Builder, and Laravel’s Filament builder are only now popularising. Preserving it in a future rebuild is not just about nostalgia — it is about keeping the platform’s most valuable competitive advantage: the ability to compose rich, interactive pages without writing code.